

Data Analytics · Python Lab Exercise

Scikit-learn & Machine Learning — Class Practice 7.5

Scikit-learn · Estimator API · fit() · predict() · score() · Linear Regression · Boston Housing Dataset

1. Scenario

Boston Housing Market — Predictive Pricing System

You are a junior data scientist at a real estate analytics firm. The pricing team needs a machine learning model that can predict the median house price in a Boston neighbourhood based on socio-economic and structural features. Your manager has asked you to build, evaluate, and interpret a linear regression model using Python's scikit-learn library — a skill used in data science roles every day.

Reference: [pyquesthub.com](https://pyquesthub.com/building-a-linear-regression-model-in-python-a-step-by-step-guide) — Building a Linear Regression Model in Python: A Step-by-Step Guide

<https://pyquesthub.com/building-a-linear-regression-model-in-python-a-step-by-step-guide>

Your completed notebook must:

- Load the Boston Housing dataset (housing.csv) into a Pandas DataFrame using `pd.read_csv()`.
- Explore the dataset — check shape, dtypes, missing values, and descriptive statistics.
- Split the data into training and testing sets using `train_test_split()`.
- Instantiate a scikit-learn Estimator (LinearRegression) and train it using the `fit()` method.
- Generate predictions on the test set using the `predict()` method.
- Evaluate model performance using the `score()` method and error metrics (MAE, MSE, R^2).
- Visualise actual vs predicted prices in a scatter plot.
- Document every step with a Markdown cell above each Code cell.

Dataset Setup — do this BEFORE opening Jupyter

A CSV file is provided with this exercise. Save it into the same folder as your notebook:

housing.csv — 40 Boston neighbourhood records (14 columns)

Your notebook file `DA_Exercise_7_5.ipynb` and `housing.csv` must be in the same folder.

Verify by running:

```
import os; print(os.listdir('.'))
```

in a Code cell before you start.

Jupyter Setup

Launch Jupyter: `jupyter notebook` in your terminal.

Navigate to the folder containing `housing.csv`.

Create a new notebook — File > New > Notebook > Python 3 (ipykernel).

Rename it: `DA_Exercise_7_5.ipynb`

Run each Code cell with Shift+Enter. Add Markdown cells with Esc → M.

2. Scikit-learn & the Estimator API — Background

Scikit-learn (sklearn) is Python's leading machine learning library. Every model in scikit-learn follows the same Estimator API — a consistent interface built around three core methods:

Method	What it does
<code>estimator.fit(X_train, y_train)</code>	Trains the model — learns the relationship between features and target.
<code>estimator.predict(X_test)</code>	Uses the trained model to generate predictions on new data.
<code>estimator.score(X_test, y_test)</code>	Returns the R^2 coefficient of determination (1.0 = perfect fit).

Because all estimators share this interface, you only need to learn the pattern once. Swapping `LinearRegression()` for `RandomForestRegressor()` or any other estimator requires changing just one line.

The LinearRegression Estimator

Linear regression models the relationship between features (X) and a continuous target (y) by fitting a straight line:

$$y = a + b_1X_1 + b_2X_2 + \dots + b_nX_n + \epsilon$$

Symbol	Meaning
y	Dependent variable — the value we predict (MEDV: median house price in \$1,000s)
$X_1 \dots X_n$	Independent variables / features (CRIM, RM, LSTAT, etc.)
a	Intercept — the baseline value of y when all features equal zero
$b_1 \dots b_n$	Coefficients — how much y changes for a unit increase in each feature
ϵ	Error term — the residual not explained by the model

3. Dataset Overview — Boston Housing

The housing.csv file contains 40 records drawn from the Boston Housing dataset. Each row represents one Boston neighbourhood. The target variable MEDV is the value the model will learn to predict.

Column	Description
CRIM	Per-capita crime rate by town
ZN	Proportion of residential land zoned for lots over 25,000 sq ft
INDUS	Proportion of non-retail business acres per town
CHAS	Charles River dummy variable (1 if tract bounds river; 0 otherwise)
NOX	Nitric oxides concentration (parts per 10 million)
RM	Average number of rooms per dwelling
AGE	Proportion of owner-occupied units built prior to 1940
DIS	Weighted distances to five Boston employment centres
RAD	Index of accessibility to radial highways
TAX	Full-value property-tax rate per \$10,000
PTRATIO	Pupil-teacher ratio by town
B	$1000(\text{Bk} - 0.63)^2$ where Bk is the proportion of Black residents
LSTAT	Percentage of lower-status population
MEDV	Median value of owner-occupied homes in \$1,000s ← TARGET

4. Requirements

Your notebook must satisfy all seven requirements below. Each maps to one or more Code cells, each preceded by a Markdown cell.

Req 1	Notebook Header — Markdown Cell
	Add a Markdown cell at the very top containing: <code># Data Analytics — Exercise 7.5: Linear Regression with Scikit-learn</code> <code>## Your Name Date</code>
	A brief (2-sentence) description of the business problem this notebook solves.

Req 2	Install Libraries & Load Dataset
	Install required packages (in terminal, not notebook): <code>pip install numpy pandas matplotlib scikit-learn</code>
	First Code cell — import all libraries: <code>import pandas as pd</code> <code>import numpy as np</code>

	import matplotlib.pyplot as plt
	from sklearn.model_selection import train_test_split
	from sklearn.linear_model import LinearRegression
	from sklearn import metrics
	Load the dataset: boston = pd.read_csv('housing.csv')
	Print: head(), shape, dtypes, and isnull().sum().

Req 3	Explore and Inspect the Dataset
	Print boston.describe() to view numeric summary statistics.
	Print the correlation of all features with MEDV: boston.corr()['MEDV'].sort_values()
	Identify the two features most positively and most negatively correlated with MEDV.
	Write a Markdown cell explaining which features appear most influential.

Req 4	Split the Dataset — train_test_split()
	Separate features and target:
	X = boston.drop('MEDV', axis=1) y = boston['MEDV']
	Split 80% train / 20% test: X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42) Print the shape of X_train, X_test, y_train, y_test.

Req 5	Instantiate the Estimator & Train with fit()
	Instantiate the LinearRegression estimator: model = LinearRegression()
	Train the model using the fit() method: model.fit(X_train, y_train) After fitting, print the model intercept and coefficients: print('Intercept:', model.intercept_)
	print('Coefficients:', model.coef_) In a Markdown cell, explain what the fit() method did in plain English.

Req 6	Generate Predictions with predict() & Evaluate with score()
	Generate predictions: y_pred = model.predict(X_test)
	Evaluate using the score() method: r2_score = model.score(X_test, y_test)
	Also compute MAE, MSE, and R ² using sklearn.metrics:

	<pre>mae = metrics.mean_absolute_error(y_test, y_pred) mse = metrics.mean_squared_error(y_test, y_pred)</pre>
	<pre>r2 = metrics.r2_score(y_test, y_pred)</pre> Print all three metrics formatted to 4 decimal places.
	In a Markdown cell, explain the difference between <code>score()</code> and <code>r2_score()</code> .

Req 7	Visualise Actual vs Predicted Prices
	Create a scatter plot of <code>y_test</code> vs <code>y_pred</code> .
	Add a red diagonal line representing perfect predictions. Label axes: 'Actual Prices (\$1,000s)' and 'Predicted Prices (\$1,000s)'.
	Add a title: 'Boston Housing — Actual vs Predicted Prices'.
	Add <code>plt.show()</code> to display the chart. Write a Markdown Summary cell (3 sentences) interpreting the model's accuracy.

5. Hints

Generic Scikit-learn Pattern	Your Task (Boston Housing)
<code>from sklearn.linear_model import LinearRegression</code>	<code>from sklearn.linear_model import LinearRegression</code>
<code>model = LinearRegression()</code>	<code>model = LinearRegression()</code>
<code>model.fit(X_train, y_train)</code>	<code>model.fit(X_train, y_train)</code>
<code>y_pred = model.predict(X_test)</code>	<code>y_pred = model.predict(X_test)</code>
<code>model.score(X_test, y_test)</code>	<code>model.score(X_test, y_test) # R² on test set</code>
<code>metrics.mean_absolute_error(y_test, y_pred)</code>	<code>metrics.mean_absolute_error(y_test, y_pred)</code>

Hint 1 — Loading the dataset

```
boston = pd.read_csv('housing.csv')
```

Ensure `housing.csv` is in the same directory as your notebook. Check with:

```
import os; print(os.listdir('.'))
```

Expected shape: (40, 14) — 40 rows, 14 columns including MEDV.

Hint 2 — Understanding the Estimator API

Think of the estimator as a 3-step workflow:

- Instantiate: `model = LinearRegression()` ← create the empty model
- Train: `model.fit(X_train, y_train)` ← teach the model on training data

- Use: `model.predict(X_test)` ← apply the learned model to new data

The `score()` method is a shortcut: it calls `predict()` internally then computes R^2 .

Hint 3 — Evaluating the model

Three complementary metrics:

- **MAE (Mean Absolute Error):** Average absolute difference between predicted and actual values. Easy to interpret — same units as MEDV (\$1,000s).
- **MSE (Mean Squared Error):** Penalises large errors more heavily. Harder to interpret directly but useful for comparing models.
- **R^2 (R-squared):** Proportion of variance in y explained by the model. 1.0 = perfect; 0.0 = no better than the mean. `model.score()` returns this.

Hint 4 — Scatter plot template

```
plt.scatter(y_test, y_pred, alpha=0.7)
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()],
         'r-', lw=2)
plt.xlabel('Actual Prices ($1,000s)')
plt.ylabel('Predicted Prices ($1,000s)')
plt.title('Boston Housing — Actual vs Predicted Prices')
plt.show()
```

Points close to the red line indicate accurate predictions.

6. Solution

Attempt all seven requirements in your notebook before reading the solution.

DA_Exercise_7_5.ipynb — complete notebook cell sequence

CELL 1 — Markdown cell (Esc → M)

```
# Data Analytics — Exercise 7.5: Linear Regression with Scikit-learn
## Your Name | Date: 2025-01-01
```

This notebook loads the Boston Housing dataset, trains a scikit-learn `LinearRegression` model using the Estimator API (`fit` / `predict` / `score`), and evaluates model performance using MAE, MSE, and R^2 .

CELL 2 — Markdown cell

```
## 1. Import Libraries and Load Dataset
```

CELL 3 — Code cell

```
import pandas as pd
```

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn import metrics

boston = pd.read_csv('housing.csv')
print('— Dataset Loaded —')
print(boston.head())
print(f'Shape: {boston.shape}')
print(boston.dtypes)
print('Missing values:')
print(boston.isnull().sum())
```

CELL 4 — Markdown cell

```
## 2. Explore and Inspect the Dataset
```

CELL 5 — Code cell

```
print('— Descriptive Statistics —')
print(boston.describe().round(2))

print('\n— Correlation with MEDV (target) —')
print(boston.corr()['MEDV'].sort_values())
```

CELL 6 — Markdown cell

```
## 3. Split the Dataset (80% Train / 20% Test)
```

CELL 7 — Code cell

```
X = boston.drop('MEDV', axis=1)    # features (13 columns)
y = boston['MEDV']                  # target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

print(f'X_train shape: {X_train.shape}')
print(f'X_test  shape: {X_test.shape}')
print(f'y_train shape: {y_train.shape}')
print(f'y_test  shape: {y_test.shape}')
```

CELL 8 — Markdown cell

```
## 4. Instantiate the Estimator and Train with fit()
```

CELL 9 — Code cell

```
# Step 1 – Instantiate the Estimator
model = LinearRegression()

# Step 2 – Train the model (the Estimator API fit() method)
model.fit(X_train, y_train)

print('— Model Trained —')
print(f'Intercept : {model.intercept_:.4f}')
coef_df = pd.DataFrame({'Feature': X.columns, 'Coefficient': model.coef_})
print(coef_df.sort_values('Coefficient',
                          ascending=False).to_string(index=False))
```

CELL 10 — Markdown cell

```
## 5. Generate Predictions with predict() and Evaluate with score()
```

CELL 11 — Code cell

```
# Step 3 – Make predictions using predict()
y_pred = model.predict(X_test)

# score() = R2 directly from the estimator
r2_from_score = model.score(X_test, y_test)

# Metrics via sklearn.metrics
mae = metrics.mean_absolute_error(y_test, y_pred)
mse = metrics.mean_squared_error(y_test, y_pred)
r2 = metrics.r2_score(y_test, y_pred)

print('— Model Evaluation —')
print(f'MAE : {mae:.4f} ($1,000s average error)')
print(f'MSE : {mse:.4f}')
print(f'R2 (metrics) : {r2:.4f}')
print(f'R2 (score()) : {r2_from_score:.4f} ← same result')
```

CELL 12 — Markdown cell

```
## 6. Visualise Actual vs Predicted Prices
```

CELL 13 — Code cell

```
plt.figure(figsize=(8, 6))
plt.scatter(y_test, y_pred, alpha=0.7, color='steelblue',
            edgecolors='white')
plt.plot([y_test.min(), y_test.max()],
         [y_test.min(), y_test.max()], 'r-', lw=2, label='Perfect
prediction')
plt.xlabel('Actual Prices ($1,000s)')
plt.ylabel('Predicted Prices ($1,000s)')
plt.title('Boston Housing – Actual vs Predicted Prices')
plt.legend()
```



```
plt.tight_layout()
plt.show()
```

CELL 14 — Markdown cell (Summary)

```
## Summary

- The LinearRegression model achieved an R² of approximately 0.72, meaning
  it explains ~72% of the variance in median house prices on the test set.
- The MAE of ~3.5 indicates that predictions are on average $3,500 from
  actual values.
- LSTAT (lower-status population %) and RM (average rooms) showed the
  strongest
  correlations with MEDV, consistent with the coefficients learned by
  fit().
```

7. Expected Output

Cell	Key Output
Cell 3 — Load data	Shape: (40, 14) missing: 0 all dtypes numeric
Cell 5 — Explore	Correlation: LSTAT most negative, RM most positive with MEDV
Cell 7 — Split	X_train (32, 13) X_test (8, 13) y_train (32,) y_test (8,)
Cell 9 — fit()	Intercept printed; 13 feature coefficients displayed
Cell 11 — predict()/score()	MAE ≈ 3.5 R² from score() matches r2_score()
Cell 13 — Plot	Scatter plot with red diagonal reference line displayed

8. Scikit-learn Quick Reference

Concept / Method	Syntax & Purpose
Import estimator	from sklearn.linear_model import LinearRegression
Import splitter	from sklearn.model_selection import train_test_split
Import metrics	from sklearn import metrics
Instantiate	model = LinearRegression() ← creates an untrained estimator
fit()	model.fit(X_train, y_train) ← train: learns coefficients
predict()	y_pred = model.predict(X_test) ← apply model to new data
score()	model.score(X_test, y_test) ← returns R² (shortcut)
Intercept	model.intercept_ ← baseline value of y
Coefficients	model.coef_ ← one weight per feature

MAE	<code>metrics.mean_absolute_error(y_test, y_pred)</code>
MSE	<code>metrics.mean_squared_error(y_test, y_pred)</code>
R ²	<code>metrics.r2_score(y_test, y_pred)</code> — 1.0 = perfect fit
<code>train_test_split</code>	<code>train_test_split(X, y, test_size=0.2, random_state=42)</code>
Drop column	<code>boston.drop('MEDV', axis=1)</code> ← creates feature matrix X
Correlation	<code>boston.corr()['MEDV'].sort_values()</code> ← feature ranking

9. Extension Challenges

1. Add a Code cell that trains the model using only the single most correlated feature (RM) rather than all 13. Compare the single-feature R² to the full-model R² and explain the difference in a Markdown cell.
2. Add a Code cell that prints a residuals plot: `plt.scatter(y_pred, y_test - y_pred)`. A good model shows residuals randomly scattered around zero. Describe what patterns (if any) you observe.
3. Add a Code cell that tests three different `test_size` values (0.1, 0.2, 0.3) in a loop, printing MAE and R² for each split. Discuss how split size affects model evaluation.
4. Add a Code cell that uses `model.coef_` to create a DataFrame of feature names and coefficients, sorted by absolute value. Which feature has the largest positive impact on predicted price?
5. Swap `LinearRegression()` for `sklearn.linear_model.Ridge(alpha=1.0)`. Rerun the full fit/predict/score pipeline and compare MSE and R² to the original model. Add a Markdown cell explaining what Ridge regression adds.